Human Verification: `Containers.Sourcing`

Sourcing as a Functor from Routed BOMs to the Opposite Network

Machine-generated from `lean/Sourcing.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Containers.Sourcing` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean. The purpose is to let a human verifier confirm that the formalisation captures the intended mathematics.

The file verifies Construction `con:sourcing` (§6 of the paper): the sourcing assignment — "each product is made somewhere, each component occurrence travels an inbound route to its point of use" — is a covariant functor of directed containers from the directed container of *routed BOMs* to the *opposite shipping network*. The file ends with the UAE steel rebar BOM as a worked example.

A formalization caveat (flagged to Robin and Neil): the paper's domain is *all* product-labelled trees, but for a general tree the sourcing position map requires routes that may not exist. The fix adopted here is to carry routing choices *on the shapes*: a routed BOM bundles each BOM edge with the inbound route of its component.

Contents

1 Overview **2**

2 The opposite digraph **2**
2.1 Definition: `Digraph.op` . . . 2

3 Routed BOMs **2**
3.1 Inductive type: `Bom` . . . 2
3.2 Inductive type: `Bom.Occ` . . . 3
3.3 Definitions: `Bom.sub` and `Bom.shift` . . . 3
3.4 Theorems: `Bom.shift_root`, `Bom.sub_shift`, `Bom.shift_assoc` . . . 4

4 The BOM directed container **5**
4.1 Definition: `bom` . . . 5

5 The sourcing functor **5**
5.1 Definition: `Bom.routeOf` . . . 5
5.2 Theorem: `Bom.routeOf_shift` (Consolidation) . . . 6
5.3 Definition: `sourcing` . . . 6

6 UAE steel instance **7**
6.1 Products, sites, and graph . . . 7
6.2 Sanity checks (examples checked by `rfl`) . . . 7

7 Translation conventions 8

1 Overview

The **opposite digraph** G^{op} of G swaps source and target: a path in G^{op} from v to u is a path in G from u to v. Positions of $\mathcal{N}(G^{\mathrm{op}})$ at v are *inbound routes* — paths of G ending at v.

A **routed BOM** over products P and graph G, indexed by its producing facility v, is a product-labelled tree whose every edge carries the inbound route (in G^{op}) of that component to its parent. Occurrences navigate this tree; the sourcing functor maps each occurrence to its total inbound route (by appending the routes along the path from root to occurrence).

2 The opposite digraph

2.1 Definition: `Digraph.op`

Definition 2.1 (Opposite digraph)**.** The **opposite** of a digraph G is the digraph G^{op} with the same nodes and edges, but with source and target swapped: $G^{\mathrm{op}}.\mathrm{src} = G.\mathrm{tgt}$ and $G^{\mathrm{op}}.\mathrm{tgt} = G.\mathrm{src}$.

Positions of $\mathcal{N}(G^{\mathrm{op}})$ at v are pairs (u, ρ) where ρ is a path in G from u to v — the *inbound routes* arriving at v.

Lean.

```
@[reducible] def Digraph.op (G : Digraph) : Digraph where
  V := G.V
  E := G.E
  src := G.tgt
  tgt := G.src
```

Haskell.

```
op :: Digraph v e → Digraph v e
op g = Digraph { src = tgt g, tgt = src g }
-- A path in (op g) from v to u is a G-path from u to v.
```

3 Routed BOMs

3.1 Inductive type: `Bom`

Definition 3.1 (Routed BOM tree)**.** A **routed BOM** $\mathtt{Bom}(P, G, v)$ over products P and graph G, indexed by producing facility v, is an inductive tree:

$$\mathtt{node}(\mathrm{prod},\ v,\ n,\ \mathrm{sites},\ \mathrm{routes},\ \mathrm{children})$$

where:

- $\mathrm{prod} : P$ is the product label at this node;
- $v : G.V$ is the facility producing `prod`;
- $n : \mathbb{N}$ is the number of direct components;
- $\mathrm{sites} : \mathrm{Fin}(n) \to G.V$ gives the producing facility of each component;
- $\mathrm{routes}(i) : \mathrm{Path}_{G^{\mathrm{op}}}(v, \mathrm{sites}(i))$ is the inbound route of component i (a G-path from $\mathrm{sites}(i)$ to v, reversed);

- children(i) : $\texttt{Bom}(P, G, \text{sites}(i))$ is the sub-BOM for component i.

Raw materials are nodes with $n = 0$ (no children).

Lean.
```
inductive Bom (P : Type) (G : Digraph) : G.V → Type where
  | node (prod : P) (v : G.V) (n : Nat) (sites : Fin n → G.V)
      (routes : (i : Fin n) → Digraph.Path G.op v (sites i))
      (children : (i : Fin n) → Bom P G (sites i)) : Bom P G v
```

Haskell.
```
data Bom p v e = Node
  { prodLabel :: p
  , site      :: v
  , children  :: [(v, [e], Bom p v e)]
    -- each child: (site, inbound route, sub-BOM)
  }
```

Fidelity note. In Lean, `Bom P G v` is indexed by the producing facility v, so the type system enforces that routes and children are consistently sited. In Haskell, v is a type parameter (not an index), so this constraint is a runtime property only.

3.2 Inductive type: `Bom.Occ`

Definition 3.2 (Occurrence)**.** An **occurrence** in a routed BOM $t : \texttt{Bom}(P, G, v)$ is a descending path from the root:

- $\texttt{here} : \texttt{Occ}(t)$ — the root occurrence;
- $\texttt{inside}(i, p) : \texttt{Occ}(t)$ where $t = \texttt{node}(\ldots, \text{children})$, $i : \text{Fin}(n)$, and $p : \texttt{Occ}(\text{children}(i))$ — an occurrence inside the i-th child.

Lean.
```
inductive Occ : {v : G.V} → Bom P G v → Type where
  | here   {v : G.V} {t : Bom P G v} : Occ t
  | inside {prod : P} {v : G.V} {n : Nat} {sites : Fin n → G.V}
      {routes : (i : Fin n) → Digraph.Path G.op v (sites i)}
      {children : (i : Fin n) → Bom P G (sites i)}
      (i : Fin n) (p : Occ (children i)) :
      Occ (.node prod v n sites routes children)
```

Haskell.
```
data Occ = Here | Inside Int Occ
  -- Here: root occurrence
  -- Inside i p: occurrence p in the i-th child
```

3.3 Definitions: `Bom.sub` and `Bom.shift`

Definition 3.3 (Sub-BOM and occurrence shift)**.** Given $t : \texttt{Bom}(P, G, v)$ and $p : \texttt{Occ}(t)$:

- $\texttt{sub}(t, p) : (u : G.V) \times \texttt{Bom}(P, G, u)$ is the sub-BOM at occurrence p (with its producing facility).
- $\texttt{shift}(t, p, q) : \texttt{Occ}(t)$ for $q : \texttt{Occ}(\texttt{sub}(t, p).\mathbf{2})$ is the concatenation: an occurrence q inside the sub-BOM at p, seen from the root of t.

Lean.
```
def sub : {v : G.V} → (t : Bom P G v) → Occ t
    → (u : G.V) * Bom P G u
  | v, t, .here ⇒ ⟨v, t⟩
```

```
  | _, .node _ _ _ _ _ children, .inside i p ⇒ (children i).sub p

def shift : {v : G.V} → (t : Bom P G v) → (p : Occ t)
    → Occ (t.sub p).2 → Occ t
  | _, _, .here, q ⇒ q
  | _, .node _ _ _ _ _ children, .inside i p, q ⇒
      .inside i ((children i).shift p q)
```

Haskell.

```
subBom :: Bom p v e → Occ → (v, Bom p v e)
subBom t Here = (site t, t)
subBom (Node _ _ children) (Inside i p) =
  subBom (thd (children !! i)) p

shift :: Bom p v e → Occ → Occ → Occ
shift _ Here q = q
shift (Node _ _ children) (Inside i p) q =
  Inside i (shift (thd (children !! i)) p q)
```

3.4 Theorems: `Bom.shift_root`, `Bom.sub_shift`, `Bom.shift_assoc`

Theorem 3.4 (BOM directed-container laws)**.** For a routed BOM $t : \mathtt{Bom}(P, G, v)$:

1. **D3** (`shift_root`): $\mathtt{shift}(t, p, \mathtt{here}) = p$ for all p.
2. **D4** (`sub_shift`): $\mathtt{sub}(t, \mathtt{shift}(t, p, q)) = \mathtt{sub}(\mathtt{sub}(t, p).\mathbf{2}, q)$ (sub-BOMs nest along the shift — multi-level costing is coherent).
3. **D5** (`shift_assoc`): the shift is associative (modulo transport along D4).

Lean.

```
theorem shift_root : {v : G.V} → (t : Bom P G v) → (p : Occ t) →
    t.shift p .here = p

theorem sub_shift : {v : G.V} → (t : Bom P G v) → (p : Occ t) →
    (q : Occ (t.sub p).2) → t.sub (t.shift p q) = (t.sub p).2.sub q

theorem shift_assoc : {v : G.V} → (t : Bom P G v) → (p : Occ t) →
    (q : Occ (t.sub p).2) →
    (r : Occ ((t.sub p).2.sub q).2) →
    t.shift (t.shift p q) ((t.sub_shift p q).symm cast r)
      = t.shift p ((t.sub p).2.shift q r)
```

Haskell.

```
-- D3: shift t p Here = p
-- D4: sub (shift t p q) = sub (sub t p) q
-- D5: shift (shift t p q) r = shift t p (shift (sub t p) q r)
```

4 The BOM directed container

4.1 Definition: `bom`

Definition 4.1 (BOM directed container)**.** The directed container $\texttt{bom}(P, G)$ has:

$$\begin{aligned}\text{Shape} &= \textstyle\sum_{v:G.V} \texttt{Bom}(P, G, v),\\ \text{Pos}(v, t) &= \texttt{Occ}(t),\\ \text{root}(v, t) &= \texttt{here},\\ \text{sub}((v, t), p) &= t.\texttt{sub}(p),\\ \text{shift}((v, t), p, q) &= t.\texttt{shift}(p, q).\end{aligned}$$

This is exactly Construction `con:bom`'s directed container of rooted trees, with labels, sites, and routes riding along the occurrences.

Lean.

```
def bom (P : Type) (G : Digraph) : DirectedContainer where
  Shape := (v : G.V) * Bom P G v
  Pos s := Bom.Occ s.2
  root _ := .here
  sub s p := s.2.sub p
  shift s p q := s.2.shift p q
  d1 _ := rfl
  d4 s p q := s.2.sub_shift p q
  d2 _ _ := rfl
  d3 s p := s.2.shift_root p
  d5 s p q r := s.2.shift_assoc p q r
```

Haskell.

```
-- The BOM DC: shapes are rooted BOM trees (with facility),
-- positions are occurrences.
data BomDC p v e = BomDC
  { bomShape :: (v, Bom p v e)
  , bomPos   :: Occ
  }
```

5 The sourcing functor

5.1 Definition: `Bom.routeOf`

Definition 5.1 (Inbound route of an occurrence)**.** The **inbound route** $\texttt{routeOf}(t, p)$ of occurrence p in BOM t at facility v is a path in G^{op} from v to the sub-BOM's facility, built by appending the routes along the path from root to p:

- $\texttt{routeOf}(t, \texttt{here}) = \mathrm{nil}_{G^{\mathrm{op}}}(v)$ (empty inbound route);
- $\texttt{routeOf}(\texttt{node}(\ldots, \text{routes}, \text{children}), \texttt{inside}(i, p)) = \text{routes}(i) +\!\!+ \texttt{routeOf}(\text{children}(i), p)$.

Lean.

```
def routeOf : {v : G.V} → (t : Bom P G v) → (p : Occ t) →
    Digraph.Path G.op v (t.sub p).1
  | v, _, .here ⇒ Digraph.Path.nil (G := G.op) v
  | _, .node _ _ _ _ routes children, .inside i p ⇒
      (routes i).append ((children i).routeOf p)
```

Haskell.

```haskell
routeOf :: Bom p v e → Occ → [e]
routeOf _ Here = []
routeOf (Node _ _ children) (Inside i p) =
  route (children !! i) ++ routeOf (thd (children !! i)) p
  where route (_, r, _) = r
```

5.2 Theorem: `Bom.routeOf_shift` (Consolidation)

Theorem 5.2 (Consolidation — F2 of the sourcing functor)**.** For all $t : \mathtt{Bom}(P, G, v)$, $p : \mathtt{Occ}(t)$, and $q : \mathtt{Occ}(\mathtt{sub}(t, p).\mathbf{2})$:

$$(\mathtt{sub}(t, \mathtt{shift}(t, p, q)).\mathbf{1},\ \mathtt{routeOf}(t, \mathtt{shift}(t, p, q))) \\ = ((\mathtt{sub}(t, p).\mathbf{2}.\mathtt{sub}(q)).\mathbf{1},\ \mathtt{routeOf}(t, p) +\!\!+ \mathtt{sub}(t, p).\mathbf{2}.\mathtt{routeOf}(q)).$$

The route of a concatenated occurrence is the concatenation of the routes. Operationally: ore reaches the rolling mill *only via* the reduction plant and the melt shop. This is the key coherence law (F2) for the sourcing functor.

Lean.

```lean
theorem routeOf_shift : {v : G.V} → (t : Bom P G v) → (p : Occ t) →
    (q : Occ (t.sub p).2) →
    (⟨(t.sub (t.shift p q)).1, t.routeOf (t.shift p q)⟩
        : (u : G.V) * Digraph.Path G.op v u)
      = ⟨((t.sub p).2.sub q).1,
          (t.routeOf p).append ((t.sub p).2.routeOf q)⟩
```

Haskell.

```haskell
-- consolidation:
-- routeOf t (shift t p q) = routeOf t p ++ routeOf (sub t p) q
propConsolidation :: Bom p v e → Occ → Occ → Bool
propConsolidation t p q =
  routeOf t (shift t p q) == routeOf t p ++ routeOf (subBomTree t p) q
  where subBomTree t p = snd (subBom t p)
```

5.3 Definition: `sourcing`

Definition 5.3 (Sourcing functor (Construction `con:sourcing`))**.** The **sourcing functor** is a covariant directed-container functor

$$\mathtt{sourcing}(P, G) : \mathtt{DContFunctor}(\mathtt{bom}(P, G),\ \mathcal{N}(G^{\mathrm{op}}))$$

with:

- on shapes: $(v, t) \mapsto v$ (the producing facility of the root product);
- on positions: $(v, t, p) \mapsto (\mathtt{sub}(t, p).\mathbf{1},\ \mathtt{routeOf}(t, p))$ (the component's facility and its inbound route).

Laws F0 (sub compatibility) and F1 (root preservation) hold by `rfl`; F2 (shift compatibility) is `routeOf_shift`.

Lean.

```lean
def sourcing (P : Type) (G : Digraph) :
    DContFunctor (bom P G) (Digraph.network G.op) where
  onShapes s := s.1
  onPos s p := ⟨(s.2.sub p).1, s.2.routeOf p⟩
  f0 _ _ := rfl
  f1 _ := rfl
  f2 s p q := s.2.routeOf_shift p q
```

Haskell.
```haskell
sourcing :: DContFunctor (BomDC p v e) (Network v e)
sourcing = DContFunctor
  { onShapes = \(v, _) → v
  , onPos    = \(_, t) p → (fst (subBom t p), routeOf t p)
  -- F0, F1 by definition; F2 is consolidation
  }
```

6 UAE steel instance

6.1 Products, sites, and graph

Definition 6.1 (UAE steel supply chain)**.** The UAE steel supply chain (§6 of the paper) has:

- products: `orePellets`, `dri`, `scrap`, `billet`, `rebar`;
- sites: `oreBerth`, `reductionPlant`, `meltShop`, `rollingMill`;
- legs: `ob2rp` (ore berth → reduction plant), `rp2ms` (reduction plant → melt shop), `ms2rm` (melt shop → rolling mill).

The shipping graph is a directed path of length 3.

Lean.
```lean
inductive Product
  | orePellets | dri | scrap | billet | rebar

inductive Site
  | oreBerth | reductionPlant | meltShop | rollingMill

inductive Leg
  | ob2rp | rp2ms | ms2rm

@[reducible] def graph : Digraph where
  V := Site; E := Leg
  src := Leg.src; tgt := Leg.tgt
```

Haskell.
```haskell
data Product = OrePellets | Dri | Scrap | Billet | Rebar
data Site    = OreBerth | ReductionPlant | MeltShop | RollingMill
data Leg     = Ob2rp | Rp2ms | Ms2rm
```

6.2 Sanity checks (examples checked by `rfl`)

Proposition 6.2 (Sourcing maps steel shape to its facility)**.** $(\texttt{sourcing}(P, G)).\texttt{onShapes}(\texttt{steelShape}) = \texttt{rollingMill}$ — the rebar BOM is produced at the rolling mill.

Proposition 6.3 (Ore occurrence inbound route)**.** The ore-pellet occurrence in the rebar BOM is sourced by the three-leg inbound route: rolling mill ← melt shop ← reduction plant ← ore berth.

Proposition 6.4 (Scrap inbound route)**.** The scrap occurrence is sourced by the one-leg inbound route: rolling mill ← melt shop (scrap is received at the melt shop, not shipped in from a remote site).

Proposition 6.5 (Consolidation instance)**.** Sourcing the concatenated occurrence billet-then-

ore equals shifting the billet's inbound route by the ore's inbound route in the opposite network:

$$\texttt{sourcing}.f(\texttt{steelShape},\ \texttt{shift}(\texttt{rebarBom}, \texttt{billetOcc}, \texttt{oreInBillet}))$$
$$= \mathcal{N}(G^{\mathrm{op}}).\texttt{shift}(\texttt{rollingMill},\ \texttt{sourcing}.f(\texttt{steelShape}, \texttt{billetOcc}),\ \texttt{sourcing}.f(\texttt{billetBom}, \texttt{oreInBill}$$

All checked by `rfl`.

Lean.

```
example : (sourcing Product graph).onShapes steelShape = rollingMill
    := rfl

example : (sourcing Product graph).onPos steelShape oreOcc
    = ⟨oreBerth, (inb ms2rm).append ((inb rp2ms).append
          ((inb ob2rp).append
           (Digraph.Path.nil (G := graph.op) oreBerth)))⟩ := rfl

example : (sourcing Product graph).onPos steelShape scrapOcc
    = ⟨meltShop, (inb ms2rm).append
          (Digraph.Path.nil (G := graph.op) meltShop)⟩ := rfl

example : (sourcing Product graph).onPos steelShape
      (rebarBom.shift billetOcc oreInBillet)
    = (Digraph.network graph.op).shift rollingMill
        ((sourcing Product graph).onPos steelShape billetOcc)
        ((sourcing Product graph).onPos ⟨meltShop, billetBom⟩
           oreInBillet) := rfl
```

Haskell.

```
-- sourcingShape rebarShape = RollingMill
-- sourcingPos rebarShape oreOcc
--   = (OreBerth, [Ms2rm, Rp2ms, Ob2rp])  (reversed legs: inbound)
-- sourcingPos rebarShape scrapOcc
--   = (MeltShop, [Ms2rm])
-- consolidation instance: rfl in Lean
```

7 Translation conventions

1. **Indexed inductive type.** `Bom P G v` is indexed by the producing facility v. In Haskell, v is a type parameter; the indexing guarantee is lost.
2. **Opposite paths as inbound routes.** A G^{op}-path from v to u is physically a G-path from u to v (same edge list, swapped endpoints). In Haskell, we represent both as the same list type and note the direction convention.
3. **Routes on shapes (formalization caveat).** The paper's BOM shapes are bare product-labelled trees; here, they carry routing choices. See the module docstring for why this is necessary.
4. **Proof bodies omitted.** Only definitions and theorem *statements* are translated. Lean checks the proofs; the human verifier checks that the *statements* match the intended mathematics.